02 The Idea

14 MIN READ · INTERACTIVE

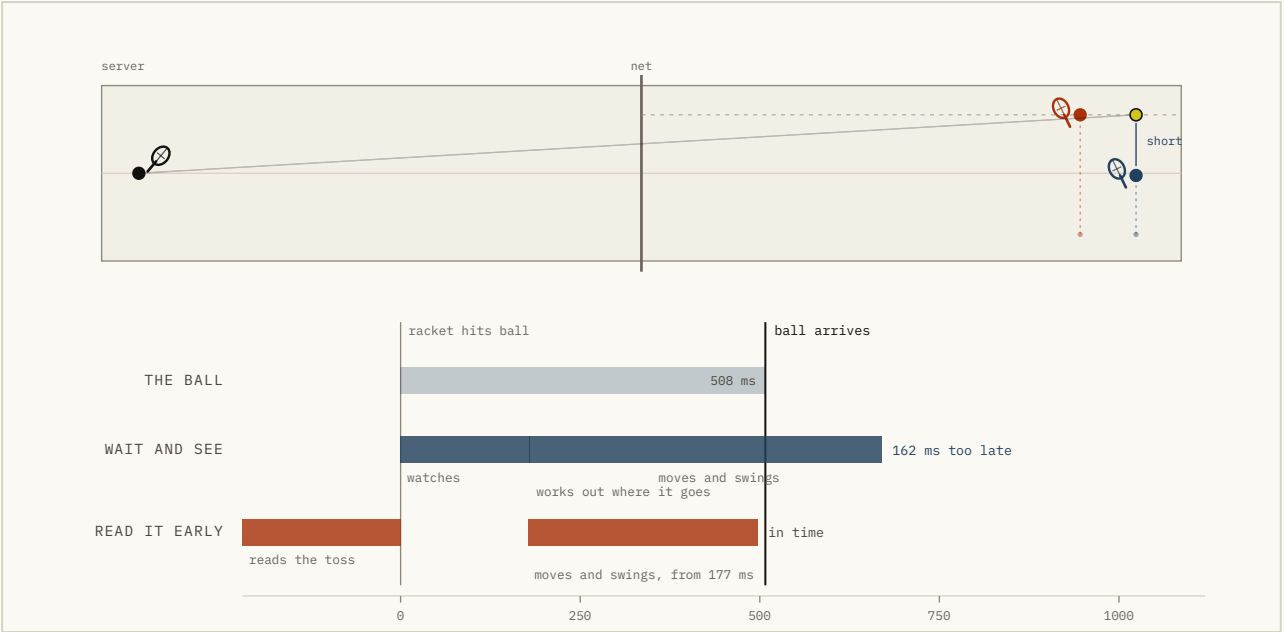
> A model you can run forward lets you try an action before paying for it. The trouble is that a search good enough to find the best plan is also good enough to find the places the model is wrong.

Tennis points start with a serve. One player throws the ball up and hits it as hard as they can. The other stands about twenty-four metres away on the far side of the net, and has to get a racket on it before it goes past. At professional level the ball leaves at something like 190 kilometres an hour, so there is roughly half a second between the two of them.

Half a second is not enough. Noticing something and beginning to move takes most of a quarter of a second before any part of you has gone anywhere, and then a whole body has to cover ground and swing.

So the good ones are not watching the ball. Work on professional play puts the earliest a movement could possibly be a response to the ball at somewhere around 140 to 160 milliseconds after the racket hits it. In one study of expert returners, the sideways movement starts at about 177 milliseconds, right on that edge. Whatever they are doing was mostly settled before contact. They read the ball toss, the shoulder and the angle of the racket face, and they move for a spot the ball has not reached.

That does not prove there is a physics engine in a tennis player's head. It proves something smaller and more useful. Past a certain speed, acting well depends on what has not happened yet.



TIME FROM RACKET TO RETURNER

508 ms

AT THIS SPEED

Too fast to watch. Only the player who had already decided arrives in time.

FIG. 2.1 Where the half second goes. Drag the serve up and the arrival line walks left into the returner's timeline. Somewhere around a hundred and forty-five the reactive plan stops fitting inside it, and the only version that still lands is the one that committed off the toss. The two returner budgets are illustrative; the 177 milliseconds is measured.

Take what you have. Run it forward. Move for what comes next.

That is a dynamics model. Of everything the phrase *world model* now gets used for, it is the oldest, and it is the one the term was coined for. What one is, what having one buys you, and the way it fails that no amount of engineering has removed.

One thing to settle first, because it trips people who go on to the papers. The model does not act. It predicts what would happen. Something else reads those predictions and picks, and that something else has its own name: the **policy** (the part that actually chooses the action, whether it does that by thinking hard or by pure reflex). Ha and Schmidhuber's own system names its third module the controller, and that module is not the world model. The split holds all the way through here.

What the model actually holds

The shape of it is not complicated. Where you are, what you do, what comes next. A dynamics model (dynamics is just the study of how things change over time, so a dynamics model is a model of what happens next) learns that, and usually learns what the step was worth as well.

The part worth slowing down on is what *where you are* is allowed to mean. It does not have to be a position and a speed. PlaNet works it out from images and never names any of it. MuZero goes further and never rebuilds the scene at all. Its model produces three numbers-ish things and no picture: how good this is, how good it will get, and what to try. The newest of them drop the picture entirely and run with no decoder (the half of a network that would turn the compressed description back into something you could look at).

$$\hat{p}(z_{t+1}, r_t \mid z_t, a_t)$$

given where you are now and an action you might take, what to expect for where you end up and what the step was worth.

FIG. 2.2 The object the whole chapter is about. Hover any symbol, or any phrase under it. The hat is the part that matters: it says *estimated*, and every failure in the second half of this chapter is that hat coming due.

So the model does not have to look like the world. It has to keep enough of what the world *does*.

> Which is why "is it photorealistic" is the wrong first question about one of these. A model of a chess position that cannot draw a bishop is still a perfectly good model of a chess position.

One prediction is not the useful part either. The useful part is that you can feed the answer back into the front. Hand it a run of actions nobody has taken, let it read its own output, and you get a future nobody has seen. That is a **rollout** (one imagined run forward: predict, feed the prediction back in, predict again), and its length is the horizon.

That is the property that does the work, and it is worth being blunt about what it is not. Not how real the thing looks. Whether you can run it forward under actions nobody took, and go shopping among them.

The idea is older than the papers usually cited for it. Thrun, Möller and Linden were already chaining a learned world model to choose future actions in 1990, in a paper with that name on the front.

Cached answers

Two cars, two identical walls.

The first brakes when the wall is nearer than some fixed distance. Somebody tuned that distance once, at the speeds they expected, and shipped it. The second carries a model of its own braking and asks, every tick, where it would stop if it braked now.

At the speed the first was tuned for, they do the same thing.

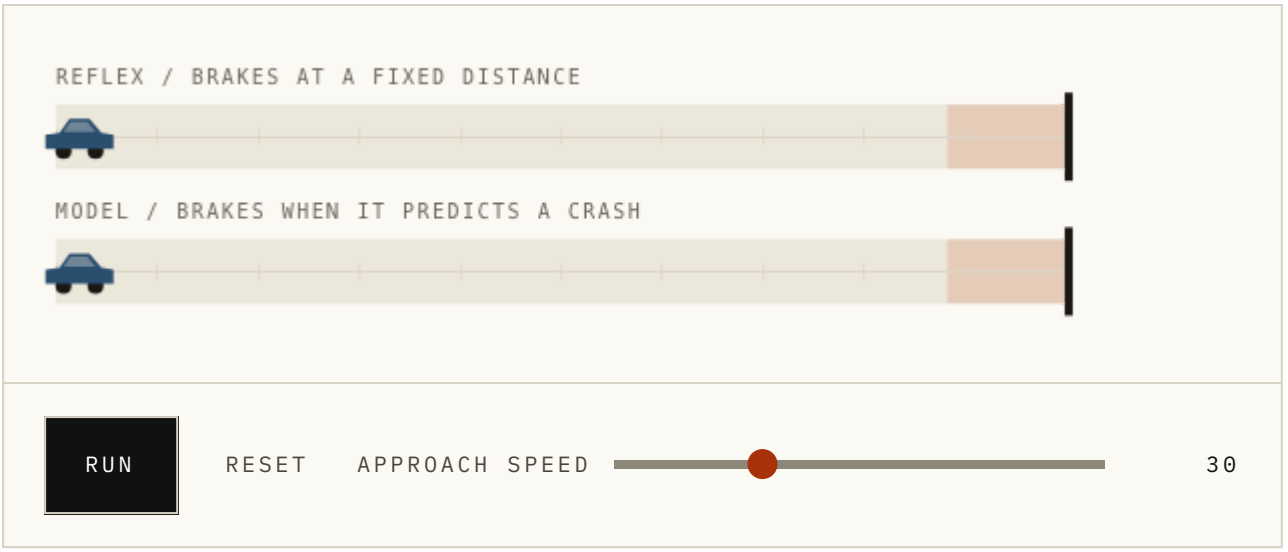


FIG. 2.3 Below about forty the two are indistinguishable, and that is the important half of the demonstration. Drag the speed up and watch which one notices. What is being compared is one fixed rule against one thing that recomputes, not model-free against model-based in general.

Be careful what that picture is comparing. A real policy with no model behind it is not a fixed rule. It can be a big network that remembers what it has already seen (a recurrent network: it hands something forward from each step to the next, so it has a memory of a sort). It changes what it does based on what it sees. It can be far cleverer than anything on screen. No model does not mean no brains.

The difference that lasts is about *when the thinking happened*. A policy did its thinking during training, and what it kept is a set of answers: in this situation, do that. A model kept something else: in this situation, if you did that, here is what you would get. The first is a cache. The second lets you work out a fresh answer once you know the actual question.

> A reflex is a *cached answer*. A model is what lets you compute a new one when the question changes. Caches are faster. You just have to know when yours has gone stale.

Caches are usually right, which is why so much of biology and so much of engineering is made of them. The interesting part starts when the question moves outside the range the answer was prepared for.

Arrive faster and the trigger still fires at the same distance. That rule holds an assumption about how long stopping takes, and nothing inside it knows it is assuming anything. The model car moves its decision because its stopping point moved and it can see that it moved.

None of which is free. The model costs computing time, has to be learned, and can be wrong. So real systems sit along a range rather than on one side of a line. Dyna mixes real experience with experience the model made up. Dreamer learns a policy inside its own imagination. MuZero pairs a learned model with search.

The question is not whether to carry a model. It is which answers to settle in advance, and which to leave until you know what you are actually facing.

What it buys you

Three things, and underneath they are the same thing. You get to work on futures that have not happened.

You can try an action before paying for it. Put the goal behind an obstacle. A **planner** (the part that tries actions out before committing to one) writes down a run of actions, plays it through the model, scores how that turned out, throws it away and writes another. PlaNet does this while driving, in its own compressed description of the scene, starting from raw pixels.

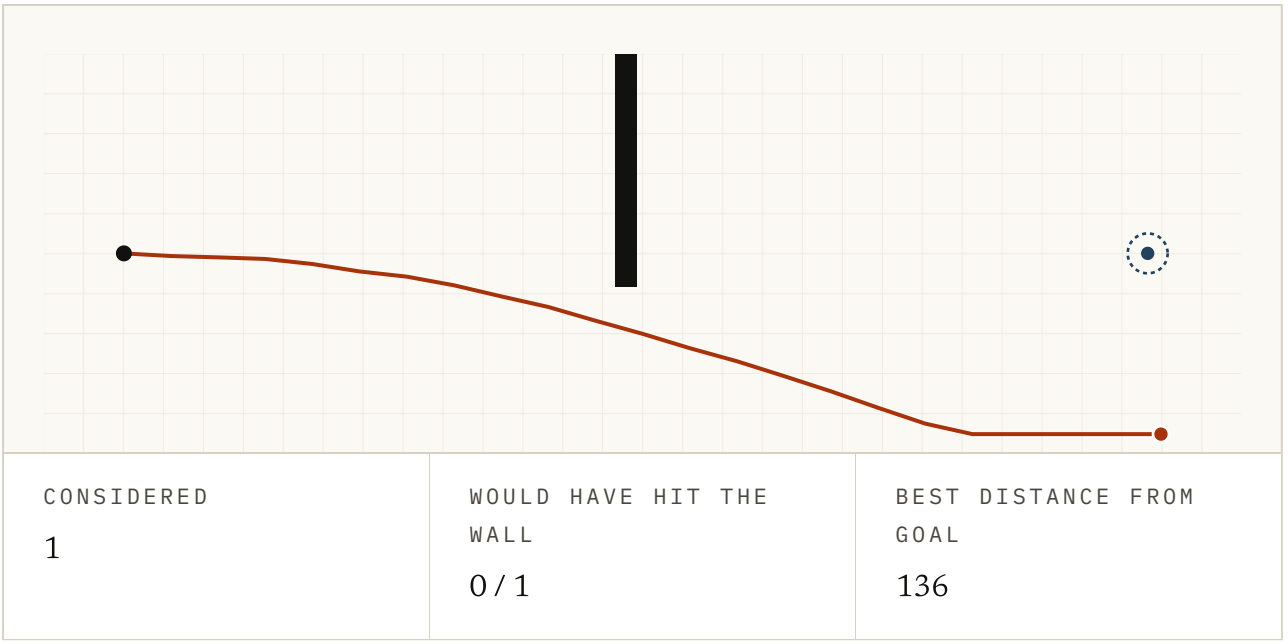


FIG. 2.4 The planner writes down runs of actions, plays each one through the same model, and keeps the best. Faint lines are the rejects. Raise the budget and the plan improves, but only for as long as the model ranks the candidates honestly. Figure 2.6 is what happens when it stops.

With one candidate it is guessing. With two hundred it is shopping. Nothing in the world changed in between. All of that came out of thinking, in the gap between seeing and moving, and a model is what makes that trade available at all.

MuZero shows how far it goes. It was never trained to redraw a game frame or lay out a chessboard, and it was never given the rules of chess, Go or shogi. It learned only the handful of quantities its search actually reads, and it played all of them well. A model does not have to reproduce everything that will happen. It has to keep whatever the decision in front of you turns on.

You can practise where mistakes are cheap. Sutton's Dyna already split it this way in 1990. Learn a model from real experience, then have the model make up more experience, and learn from that too. Dreamer put the trick at the centre. Imagine long runs, learn the behaviour from the imagined ones, and by the time it has to act there is nothing left to work out. Its third version ran that recipe across more than 150 tasks, retuned for none of them.

The tempting version of this is *reality is expensive, imagination is free*, and it is wrong in a way worth fixing. Imagining costs computing time, sometimes an enormous amount of it. What it saves is contact with the world: worn-out robots, lab hours, someone supervising, the crash you do not get to undo.

Reality charges you in experience. A model lets you pay part of the bill in computing time instead.

You can ask what if. What if I brake now, what if I turn instead, what if I push it from the other side. Comparing those without running them is the whole reason it matters that the model is told which action you mean. It moves the question from *what probably happens next* to *what happens if I do this*.

Be exact about what you get. The answer is true inside the model: given what it has learned, this is what braking earlier would have done. It is not a report from the world that did not happen. A patch of oil nobody saw, a gust, anything the model never had, and the answer is confidently wrong.

The world only ever runs the action you took. The model is where the alternatives get to exist.

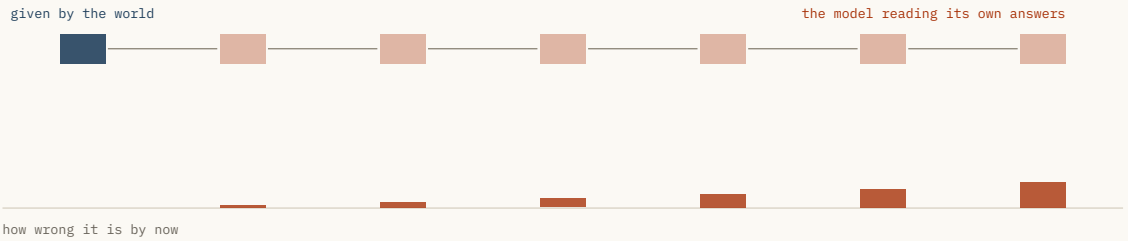
Where it goes wrong

Everything so far argues for more of it. Better model, more candidates, longer runs, pick the winner. Every one of those is reasonable on its own, and together they walk you into the failure this whole field is organised around.

Two things go wrong, and they are not the same thing.

Errors compound. A model that is slightly wrong once is fine. A model that is slightly wrong and then reads its own answer back in is fine for a while and then is nowhere near. Nothing breaks at any step. A one-step model can be accurate enough

to trust while a hundred-step fantasy stitched out of that same model is worthless. So there is a limit on how far ahead it is worth looking, and the fix in the last section is as blunt as it is because of that.



ERROR ADDED PER STEP	ERROR AFTER 6 STEPS
2%	21%

FIG. 2.5 Two per cent added per step, which is a good model. The first box is given by the world and every box after it is the model reading its own last answer, so the error does not add, it multiplies. Nothing breaks at any step.

Search goes hunting for the errors. This one is stranger, and it is the reason the chapter exists.

Say the learned map is wrong in exactly one place: it thinks there is a gap in a wall that is actually solid. Try plans at random and you will almost never go near it. A weak search never finds it. A thorough one does, because the best route in the whole learned world runs straight through it.

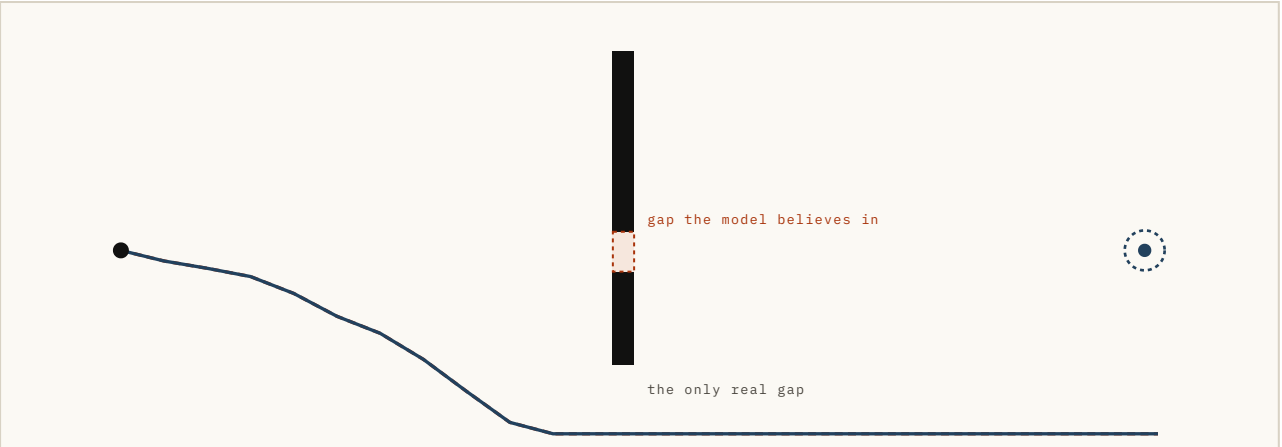


FIG. 2.6 The model believes there is a gap in the wall. There is not. Leave the effort low and the planner never finds it, takes the long way round, and gets there. Then turn the effort up. This is a failure the setup makes visible, not a law that more search always hurts.

Read that one twice, because it runs backwards through most engineering instinct. The weaker search produced a plan that worked. The stronger one produced a plan that scored better and hit a wall. It was not fighting you. It did exactly what you asked, which was to find the plan the model likes most.

The field calls this **model exploitation**. The better the planner gets at beating the model, the more it drifts toward the places the model was never taught, and those are exactly where its mistakes are. Systems that learn from a fixed pile of recorded experience (called offline: no way to go back out and collect the thing you turned out to need) have it worst, because they cannot go and get the experience that would correct them.

So a model can be wrong by almost nothing on average and still be a dangerous thing to plan inside. Average error asks how it does on the cases you measured. Control asks what happens once something starts hunting, on purpose, for the highest score it can find. Those are not the same test.

The best plan in the model and the best plan in the world are only the same plan if the model is good *where the search goes looking*. Everything in the next section is an attempt to make that last part true.

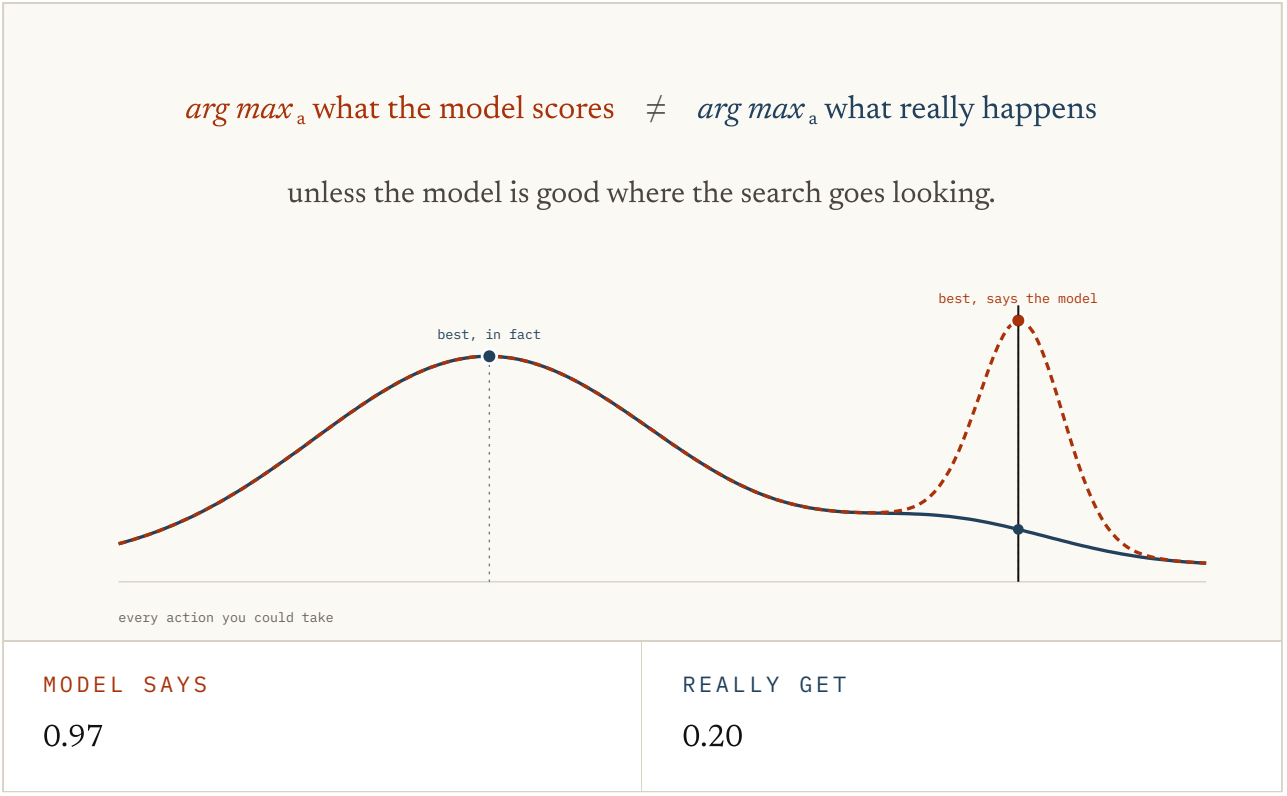


FIG. 2.7 Two ways of scoring the same actions. They agree nearly everywhere, which is what a low average error looks like. Drag across to the spike and read both numbers: the model's favourite action scores 0.97 in imagination and 0.20 in the world.

> Planning is the process of taking a model as input and producing or improving a policy for interacting with the modelled world.
>
> Sutton on planning as computation over a learned model, in the paper that set up the problem (Dyna, an Integrated Architecture for Learning, Planning and Reacting, 1991) https://mlanthology.org/icml/1990/sutton1990icml-integrated/

Exactly right, and the difficulty is sitting in *the modelled world*. What you get is a policy that is good for the world in the model. Whether that is the world you are standing in is a separate question, and nothing inside the model can answer it.

What people do about it

Nobody fixed this once and moved on. What the field has instead is a set of ways to limit how far imagination is trusted, and they are all the same instruction in different clothes.

Ask more than one model. Train several instead of one (an ensemble: same data, different starting points, so they end up disagreeing about anything the data never settled) and carry their disagreement through the plan. Where they disagree, the plan has wandered somewhere nothing supports.

Disagreement is a hint, not a verdict. Models trained on the same gaps can share a blind spot and agree, confidently, about nothing at all. And a model's own confidence is not much better: when one says it is 70 percent sure, it is right about 70 percent of the time only if somebody went and checked. Somebody did, found it was often out, and found that correcting it made the planning better.

Do not dream far. One answer is almost rude in its simplicity: start every imagined run from somewhere the world really produced, and keep it short. You lose the free infinity of simulation. You gain a limit on how long an error has to grow before real data interrupts it.

Planning on a short clock does the same job. Plan a little, act a little, look again, plan again (the standard name for this is model-predictive control, and it is most of robotics). Replanning cannot remove a mistake the model makes everywhere, but it does keep handing the world chances to say where things actually are.

Charge for not knowing. Take the score the model hands back and dock it wherever the model is unsure. An unfamiliar-looking shortcut then has to be good enough to cover the cost of nobody knowing what is down there.

Underneath all three: *do not let the planner collect full marks for going somewhere the model has not earned the right to be confident about.*

None of it makes the problem go away. The best of these systems now hold up across a startling range of tasks. But they are careful ways of using a model that is wrong. They are not proof of one that is right.

The question was never how to make the model accurate. It is: accurate where, for which actions, how far out, against how hard a search, and what should the thing do when it does not know.

Try it

1 Below the tuned speed in the braking demo, the fixed-trigger car and the model car behave identically. What does that establish?

a. The dynamics model is not doing anything
b. A cached rule can be entirely sufficient inside the conditions it was prepared for
c. Model-based control only matters at high speed
d. Model-free systems cannot use velocity

> ANSWER b. A cached rule can be entirely sufficient inside the conditions it was prepared for. The point is not that the model wins everywhere. Inside the range where a cached answer is still correct, recomputing it buys you nothing except the cost of recomputing it.

2 A recurrent policy maps observations and memory straight to actions, but holds no learned transition function you can roll forward under actions it has not taken. Is it model-based?

a. Yes, because it has memory
b. Yes, because every large network contains a model
c. No
d. Only if its policy is stochastic

> ANSWER c. No. Memory and complexity do not by themselves make a callable dynamics model. The missing contract is the ability to predict consequences under hypothetical actions.

3 A system encodes a camera frame into 512 numbers, rolls those numbers forward under 500 candidate action sequences, and picks the best. It never decodes a future image. Does it qualify?

a. No, because a world model has to predict pixels
b. Yes, because its learned state can be rolled forward under actions
c. Only if the 512 numbers correspond to named physical quantities
d. Only if the predictions are deterministic

> ANSWER b. Yes, because its learned state can be rolled forward under actions. PlaNet, MuZero and TD-MPC2 are the counterexamples to the idea that a useful world model has to reproduce the future visually. Decision-relevant latent dynamics are enough.

4 Dreamer trains an actor on trajectories generated by its world model, then the actor picks an action directly. No large search runs at that instant. Has the world model stopped counting?

a. Yes, because planning has to happen at inference time
b. No, the dynamics still generated action-conditioned imagined experience

c. Yes, unless the actor reconstructs the pixels
d. It depends only on the size of the actor

> ANSWER b. No, the dynamics still generated action-conditioned imagined experience. Online search is one use of an iterable dynamics model, not the definition of one. Dreamer spends the model earlier, during training, rather than at the moment of acting.

5 In the exploitation figure the planner gets worse after you raise its search budget. What happened?

a. The optimiser became less accurate
b. The world became harder
c. Better optimisation found a model error that weaker search had missed
d. Long plans are always worse than short plans

> ANSWER c. Better optimisation found a model error that weaker search had missed. The optimiser got better at maximising the score the model hands it. The mistake was assuming that score stayed faithful to reality everywhere the search could reach.

6 Why can short model rollouts help?

a. Neural networks cannot make more than a few predictions
b. They limit how long model error and distribution shift can accumulate before real data returns
c. Short horizons make the model exact
d. They eliminate uncertainty

> ANSWER b. They limit how long model error and distribution shift can accumulate before real data returns. This is the motivation behind MBPO's short rollouts branched from real states. Use the model enough to gain synthetic experience, not so far that accumulated bias swamps it.

7 Five models in an ensemble all agree a shortcut is safe. What has that proved?

a. The shortcut is safe
b. The probability of failure is zero
c. Only that these five agree; a shared blind spot is still possible
d. That more models were unnecessary

> ANSWER c. Only that these five agree; a shared blind spot is still possible. Disagreement is a useful uncertainty signal, which is what PETS exploits. But learned uncertainty can itself be miscalibrated, and models trained on the same biased data can be confidently wrong together.

8 Your model says: if you had braked one second earlier, you would have stopped before the wall. What exactly do you have?

a. Proof of what the physical world would truly have done
b. A model-relative prediction under a hypothetical action
c. A recording of the alternative history
d. A causal conclusion independent of hidden variables

> ANSWER b. A model-relative prediction under a hypothetical action. This is the useful sense of what if in model-based control. It lets you compare actions before taking them, and its authority reaches exactly as far as the learned model does.

FIG. 2.8 Eight questions on the distinction that matters here. Not whether the agent looks clever, but where it works out its consequences, and when that working-out is allowed to run ahead of the evidence.

All of this quietly assumed you already had a state to run forward. A position, a speed, some short list of numbers holding whatever the future turns on. Nobody hands you one. You get pixels.

Thanks for reading. Chapter 3 is about why *predict the next thing*, a goal that sounds far too simple to work, turns out to be enough to pull the structure of a world out of raw experience.

Sources

World Models HA & SCHMIDHUBER, 2018 ↗

Encoder, latent dynamics, tiny controller, and the experiments where the controller is trained inside the model's own generated environment before being moved back.

https://arxiv.org/abs/1803.10122

Learning Latent Dynamics for Planning from Pixels (PlaNet) HAFNER ET AL., 2018 ↗

Stochastic latent dynamics learned from images, then searched over at decision time. Figure 2.2 in its real form.

https://arxiv.org/abs/1811.04551

Dream to Control (Dreamer) HAFNER ET AL., 2019 ↗

Actor and critic trained on imagined latent trajectories, so no expensive search has to run at the moment of acting.

https://arxiv.org/abs/1912.01603

Mastering diverse control tasks through world models (DreamerV3)

HAFNER ET AL., NATURE 2025 ↗

One algorithm and one hyperparameter setting across more than 150 tasks. The strongest recent evidence for the imagination branch.

https://www.nature.com/articles/s41586-025-08744-2

Mastering Atari, Go, chess and shogi by planning with a learned model (MuZero)

SCHRITTWIESER ET AL., 2020 ↗

The clearest proof that planning needs no reconstruction of observations. The model learns only what the search consumes.

https://arxiv.org/abs/1911.08265

TD-MPC2: Scalable, Robust World Models for Continuous Control

HANSEN, SU & WANG, 2024 ↗

Decoder-free latent dynamics with local trajectory optimisation, scaled to a single multi-task agent across 104 control tasks.

https://arxiv.org/abs/2310.16828

Deep RL in a Handful of Trials using Probabilistic Dynamics Models (PETS)

CHUA ET AL., 2018 ↗

Ensembles and trajectory sampling: the standard attempt to make uncertainty part of the plan rather than an afterthought.

https://arxiv.org/abs/1805.12114

Dyna: an Integrated Architecture for Learning, Planning and Reacting SUTTON, 1991 ↗

Planning defined as computation over a learned model, and the loop that interleaves it with real experience.

https://mlanthology.org/icml/1990/sutton1990icml-integrated/

Recurrent world models for planning and curiosity SCHMIDHUBER, 1990 ↗

The controller and the world model kept as separate objects, which is the distinction this chapter opens on.

https://people.idsia.ch/~juergen/world-models-planning-curiosity-fki-1990.html

Planning with an Adaptive World Model THRUN, MÖLLER & LINDEN, 1990 ↗

A learned world model built through interaction and then chained to optimise future actions, twenty-eight years before the label stuck.

https://papers.nips.cc/paper_files/paper/1990/hash/9be40cee5b0eee1462c82c6964087ff9-Abstract.html

When to Trust Your Model: Model-Based Policy Optimization JANNER ET AL., 2019 ↗

Short rollouts branched from real states, as a direct answer to compounding error and exploitation. The title is the chapter's question.

https://arxiv.org/abs/1906.08253

Benchmarking Model-Based Reinforcement Learning WANG ET AL., 2019 ↗

Where model-based methods actually win and lose, and where the planning horizon dilemma gets named and measured.

https://arxiv.org/abs/1907.02057

Calibrated Model-Based Deep Reinforcement Learning MALIK ET AL., 2019 ↗

The warning underneath every uncertainty method: the uncertainty estimate can itself be wrong, and calibrating it changes planning results.

https://arxiv.org/abs/1906.08312

MOPO: Model-based Offline Policy Optimization YU ET AL., 2020 ↗

Pessimism made explicit. Penalise predicted reward by model uncertainty, so an unfamiliar shortcut has to pay for being unfamiliar.

https://arxiv.org/abs/2005.13239

Quantifying the nature of anticipation in professional tennis TRIOLET ET AL., 2013 ↗

Match analysis placing the earliest possible reaction to the ball around 140 to 160 ms after contact. Paywalled.

https://doi.org/10.1080/02640414.2012.759658

The spatiotemporal control of expert tennis players when returning first serves NAVIA ET AL., 2021 ↗

Where the 177 ms figure in the opening comes from, measured rather than estimated. Paywalled.

https://doi.org/10.1080/02640414.2021.1976484

FIG. 2.9 Ordered for someone building on this rather than for historical completeness.